The first file system's ("Old UNIX fs") data structures looked like this:

The super block (S) contained information about the entire file system: how big the volume is, how many inodes there are, a pointer to the head of a free list of blocks, and so forth. The inode region of the disk contained all the inodes for the file system. Finally, most of the disk was taken up by data blocks.

The Problem: Poor Performance

This file system was delivering only 2% of the overall disk bandwidth.

The main issue was that Old Unix file system treated the disk like it was a random-access memory.

For example, data blocks of a file were often very far away from its inode, thus inducing an expensive seek.

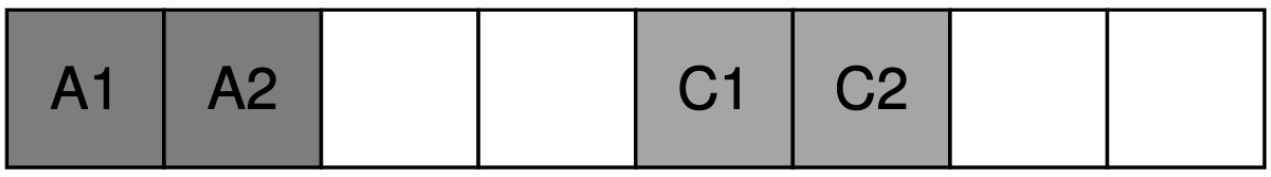
Worse, the file system would end up getting quite fragmented, as the free space was not carefully managed.

The free list would end up pointing to a bunch of blocks spread across the disk.

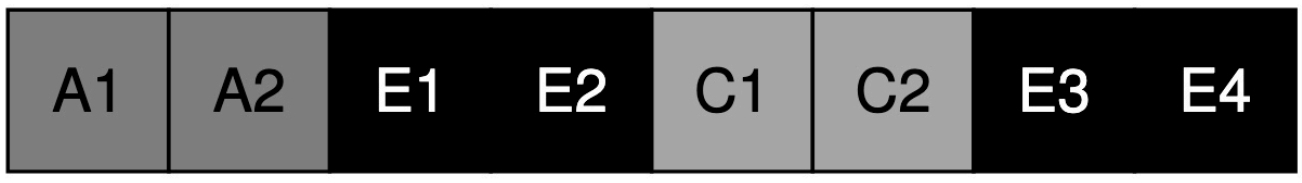
The result was that a logically contagious file would be accessed by going back & forth across the disk, thus reducing performance.

For example, imagine the following data block region which contains 4 files (A, B, C, D), each of size 2 blocks:

If B & D are deleted, the resulting layout is:



Now, the free space is fragmented into two chunks of two blocks instead of a one nice contagious chunk of four. Lets say, you wish to allocate a file E:



As a result, E is spread across the disk.

This problem is exactly what disk defragmentation tools help with. They re-organize on-disk data to place files contagiously and make free space for one or few contagious regions.

On the other hand, the orignal block was too small (512 bytes). Smaller blocks were good because they minimized internal fragmentation, but bad for transfer as each block might require positioning overhead to reach it.

THE CRUX:
HOW TO ORGANIZE ON-DISK DATA TO IMPROVE PERFORMANCE
How can we organize file system data structures so as to improve performance? What types of allocation policies do we need on top of those data structures? How do we make the file system "disk aware"?

FFS: Disk Awareness Is the Solution

A group at Berkeley decided to build, faster file system called Fast File System.

The idea was to design the file system structures & allocation policies to be "disk aware" and thus improve performance.

They kept the same interface to the file system (same API calls) but changed the internal implementation.

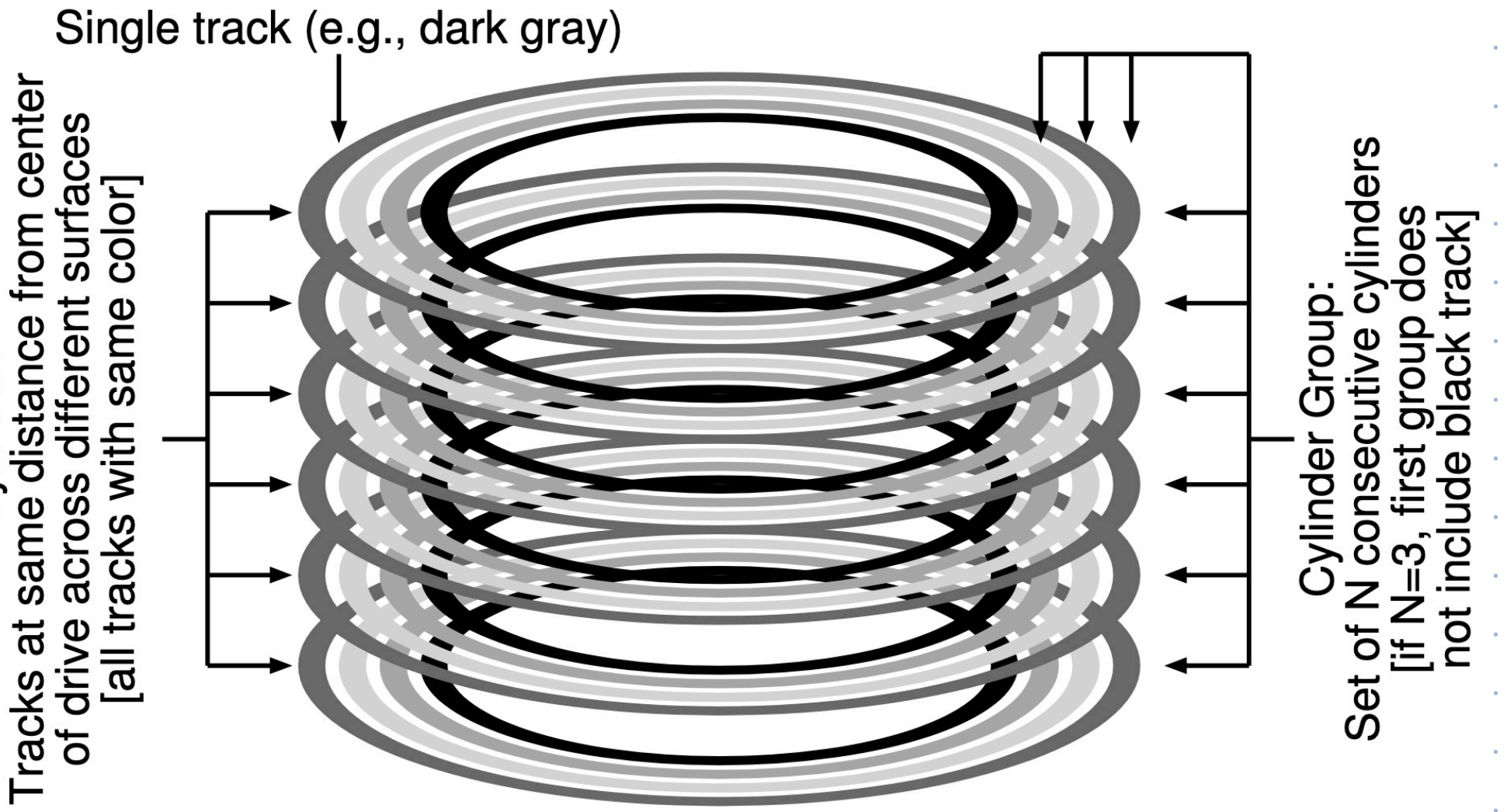
Organizing Structure: The Cylinder Group

FFS divides the disk into number of cylinder groups.

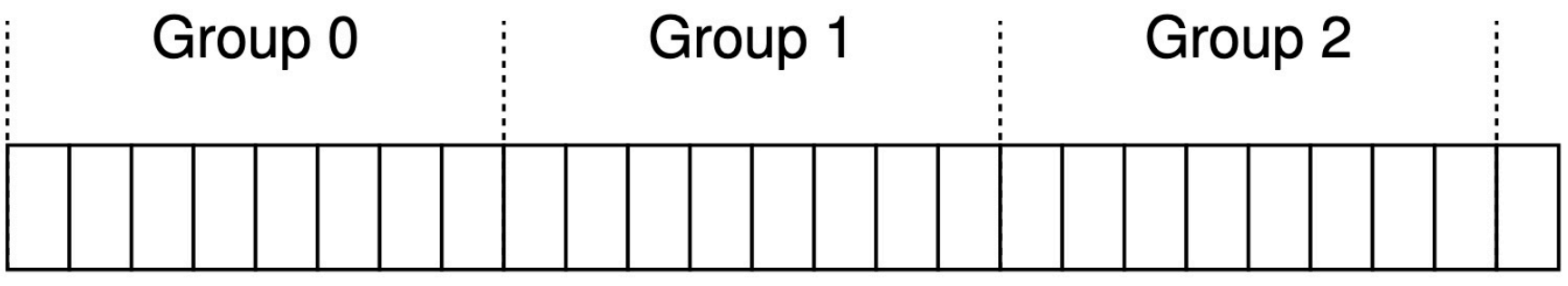
A single cylinder is a set of tracks on different surfaces of a hard drive that are the same distance from the center of the drive.

FFS aggregates N consecutive cylinders into group, and thus the entire disk can thus be viewed as a collection of cylinder groups.

Here is a simple example

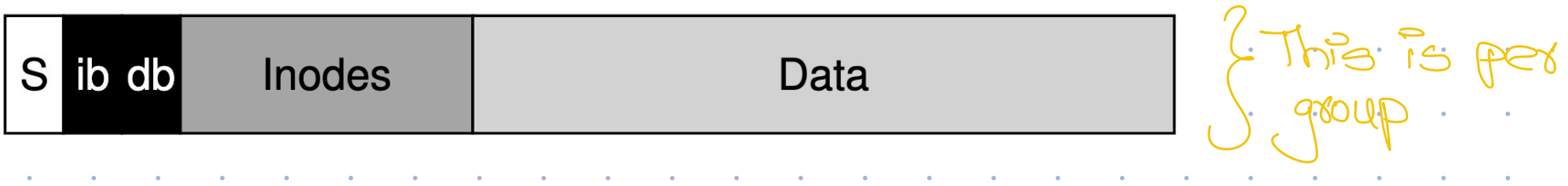


Modern drives don't export enough information for the file system to understand whether a pasticular cylinder is in use. Thus, modern systems (ext2, ex3, ext4) instead organize the drive into block groups or cylinder groups. In below image, each group has 8 blocks:



By placing two files in the same group, FFS can ensure that accessing one after the other will not result in long seeks across the disks.

To store files & directories, FFS includes space for inodes, data blocks, and some structures that track whether each of those are allocated or free within each group. Visually,



FS keeps a copy of the super block (S) in each group for reliability reasons. The super block is needed to mount the file system; by keeping multiple copies, if one copy becomes corrupt, you can still mount and access the file system by using a working replica. Within each group, FFS needs to track whether the inodes and data blocks of the group are allocated. A per-group inode bitmap (ib) and data bitmap (db) serve this role for inodes and data blocks in each group. Bitmaps are an excellent way to manage free space in a file system be- cause it is easy to find a large chunk of free space and allocate it to a file, perhaps avoiding some of the fragmentation problems of the free list in the old file system. Finally, the inode and data block regions are just like those in the previous very-simple file system (VSFS). Most of each cylinder group is comprised of data blocks.

Policies: How To Allocate Files And Directories

The basic mantra of FFS is:

"keep related stuff close" i.e place it within the same block group.

&

"keep unrelated stuff far apart" i.e place them in different block groups.

To achieve this end, FFS makes use of a few simple placement heuristics.

1. For directories, FFS employs a simple approach: find the cylinder group with a low number of allocated directories (to balance directories across groups) and a high number of free inodes (to subsequently be able to allocate a bunch of files), and put the directory data and inode in that group.

2. For files, FFS does two things. First, it makes sure (in the general case) to allocate the data blocks of a file in the same group as its inode, thus preventing long seeks between inode and data (as in the old file system). Second, it places all files that are in the same directory in the cylinder group of the directory they are in. Thus, if a user creates four files, /a/b, /a/c, /a/d, and b/f, FFS would try to place the first three near one another (same group) and the fourth far away (in some other group)

For example, assume that there are only 10 inodes & 10 data blocks in each group, and three directories (/, /a, /b) & four files (/a/c, /a/d, /a/e, /b/f) are placed within them per FFS policies.

Further, assume regular files are two data blocks in size, and directories have just a single data block. Visually,

```
group inodes     data
    0 /--------- /---------
    1 acde------ accddee---
    2 bf-------- bff-------
    3 ---------- ----------
    4 ---------- ----------
    5 ---------- ----------
    6 ---------- ----------
    7 ---------- ----------
```

FFS does two positive things: the data blocks of each file are near each file's inode, and files in the same directory are near one another

In contrast, lets look at an inode allocation policy that simply spreads inodes across groups, trying to ensure that no group's inode table fills up quickly. Visually,

```
group inodes     data
    0 /--------- /---------
    1 a--------- a---------
    2 b--------- b---------
    3 c--------- cc--------
    4 d--------- dd--------
    5 e--------- ee--------
    6 f--------- ff--------
    7 ---------- ----------
```

As you can see from the figure, while this policy does indeed keep file (and directory) data near its respective inode, files within a directory are arbitrarily spread around the disk, and thus name-based locality is not preserved. Access to files /a/c, /a/d, and /a/e now spans three groups instead of one as per the FFS approach.

FFS heuristic are based on common sense & not extensive studies.

Files in a directory are often accessed together

Measuring File Locality

To understand better whether these heuristics make sense, let's analyze some traces of file system access and see if indeed there is namespace locality.

Specifically, we'll use the SEER traces and analyze how "far away" file accesses were from one another in the directory tree. For example, if file f is opened, and then re-opened next in the trace (before any other files are opened), the distance between these two opens in the directory tree is zero (as they are the same file). If a file f in directory dir (i.e., dir/f) is opened, and followed by an open of file g in the same directory (i.e., dir/g), the distance between the two file accesses is one, as they share the same directory but are not the same file. Our distance metric, in other words, measures how far up the directory tree you have to travel to find the common ancestor of two files; the closer they are in the tree, the lower the metric.

Figure 41.1 shows the locality observed in the SEER traces over all workstations in the SEER cluster over the entirety of all traces. The graph plots the difference metric along the x-axis, and shows the cumulative percentage of file opens that were of that difference along the y-axis. Specifically, for the SEER traces (marked "Trace" in the graph), you can see that about 7% of file accesses were to the file that was opened previously, and that nearly 40% of file accesses were to either the same file or to one in the same directory (i.e., a difference of zero or one). Thus, the FFS locality assumption seems to make sense (at least for these traces). Interestingly, another 25% or so of file accesses were to files that had a distance of two. This type of locality occurs when the user has structured a set of related directories in a multi-level fashion and consistently jumps between them. For example, if a user has a src directory and builds object files (.o files) into an obj directory, and both of these directories are sub-directories of a main proj directory, a common access pattern will be proj/src/foo.c followed by proj/obj/foo.o. The distance between these two accesses is two, as proj is the common ancestor. FFS does not capture this type of locality in its policies, and thus more seeking will occur between such accesses.

For comparison, the graph also shows locality for a "Random" trace.The random trace was generated by selecting files from within an existing SEER trace in random order, and calculating the distance metric between these randomly-ordered accesses. As you can see, there is less namespace locality

in the random traces, as expected. However, because eventually every file shares a common ancestor (e.g., the root), there is some locality, and thus random is useful as a comparison point.

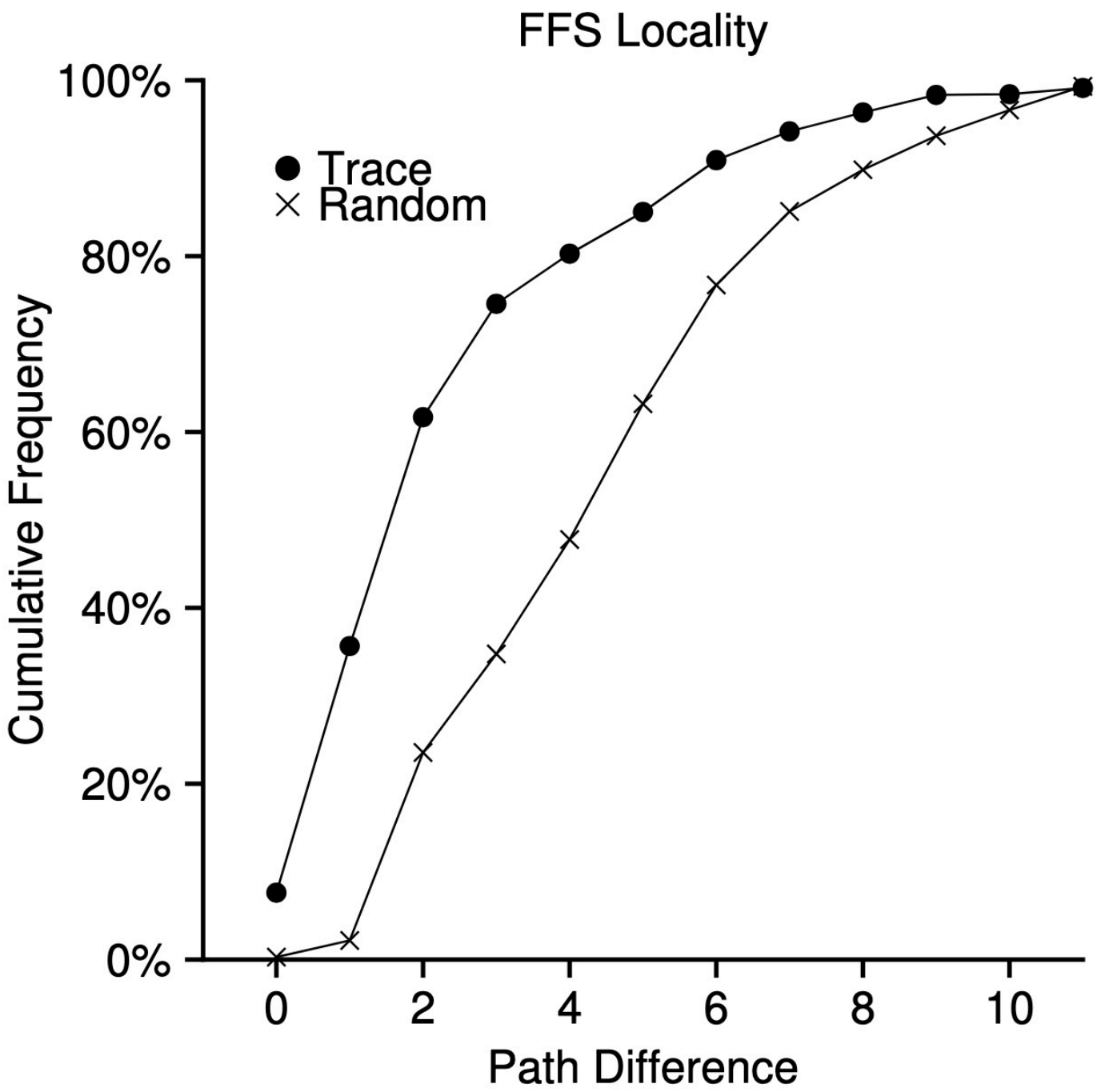


Figure 41.1: FFS Locality For SEER Traces

The Large-File Exceptions

Without an exception for large files, it would entirely fill the block group it is first placed within.

Filling a group in this manner is undesirable, as it prevents subsequent "related" files from being placed within this block group, and thus may hurt file-access locality.

Thus, for large files, FFS does the following. After some number of blocks are allocated into the first block group

FFS places the next "large" chunk of the file in another block group & so on.

Visually, without the large file exception, a file (/a) with 30 blocks is placed as:

```
group inodes     data
  0   /a-------- /aaaaaaaaa aaaaaaaaaa aaaaaaaaaa a---------
  1   ---------- ---------- ---------- ---------- ----------
  2   ---------- ---------- ---------- ---------- ----------
```

As shown, /a fills up most of the data blocks in Group 0, whereas other groups remain empty.

If some other files are now created in the root directory (/), there is not much room for their data.

With large file exception, we have:

```
group inodes     data
  0   /a-------- /aaaaa---- ---------- ---------- ----------
  1   ---------- aaaaa----- ---------- ---------- ----------
  2   ---------- aaaaa----- ---------- ---------- ----------
  3   ---------- aaaaa----- ---------- ---------- ----------
  4   ---------- aaaaa----- ---------- ---------- ----------
  5   ---------- aaaaa----- ---------- ---------- ----------
  6   ---------- ---------- ---------- ---------- ----------
```

Note that spreading blocks of a file across the disk will hurt performance, particularly in the relatively common case of sequential file access (e.g., when a user or application reads chunks 0 through 29 in order). But you can address this problem by choosing chunk size carefully. Specifically, if the chunk size is large enough, the file system will spend most of its time transferring data from disk and just a (relatively) little time seeking between chunks of the block. This process of reducing an overhead by doing more work per overhead paid is called **amortization** and is a common technique in computer systems.

Lets do an example: assume that the average positioning time (i.e seek & rotation) for a disk is 10ms. Assume further that the disk transfers data at 40 MB/s.

If your goal was to spend half our time seeking between chunks and half our time transferring data (and thus achieve 50% of peak disk performance), you would thus need to spend 10ms transferring data for every 10ms positioning.

So the question becomes: how big does a chunk have to be in order to spend 10ms in transfer?

$$\frac{40\ \cancel{MB}}{\cancel{sec}} \cdot \frac{1024\ KB}{1\ \cancel{MB}} \cdot \frac{1\ \cancel{sec}}{1000\ \cancel{ms}} \cdot 10\ \cancel{ms} = 409.6\ KB$$

To achieve 90% of peak disk performance:

$$0.9 = \frac{T_{trans}}{10 + T_{trans}}$$

$$0.9(10 + T_{trans}) = T_{trans}$$

$$9 + 0.9T_{trans} = T_{trans}$$

$$9 = 0.1T_{trans}$$

$$T_{trans} = 90ms$$

$$\frac{40MB}{\cancel{sec}} \times \frac{1\ \cancel{sec}}{1000\ \cancel{ms}} \times 90\cancel{ms} = 3.6\ MB$$

$$E = \frac{T_{trans}}{T_{pos} + T_{trans}}$$

To achieve 99% of peak disk performance:

$$0.99 = \frac{T_{trans}}{10 + T_{trans}}$$

$$0.99(10 + T_{trans}) = T_{trans}$$

$$9.9 + 0.99T_{trans} = T_{trans}$$

$$9.9 = 0.01T_{trans}$$

$$T_{trans} = 990$$

$$\frac{40MB}{\cancel{sec}} \times \frac{1\ \cancel{sec}}{1000\ \cancel{ms}} \times 990\cancel{ms} = 39.6\ MB$$

As shown in Figure 41.2, closer you get to peak, the bigger these chunks get:

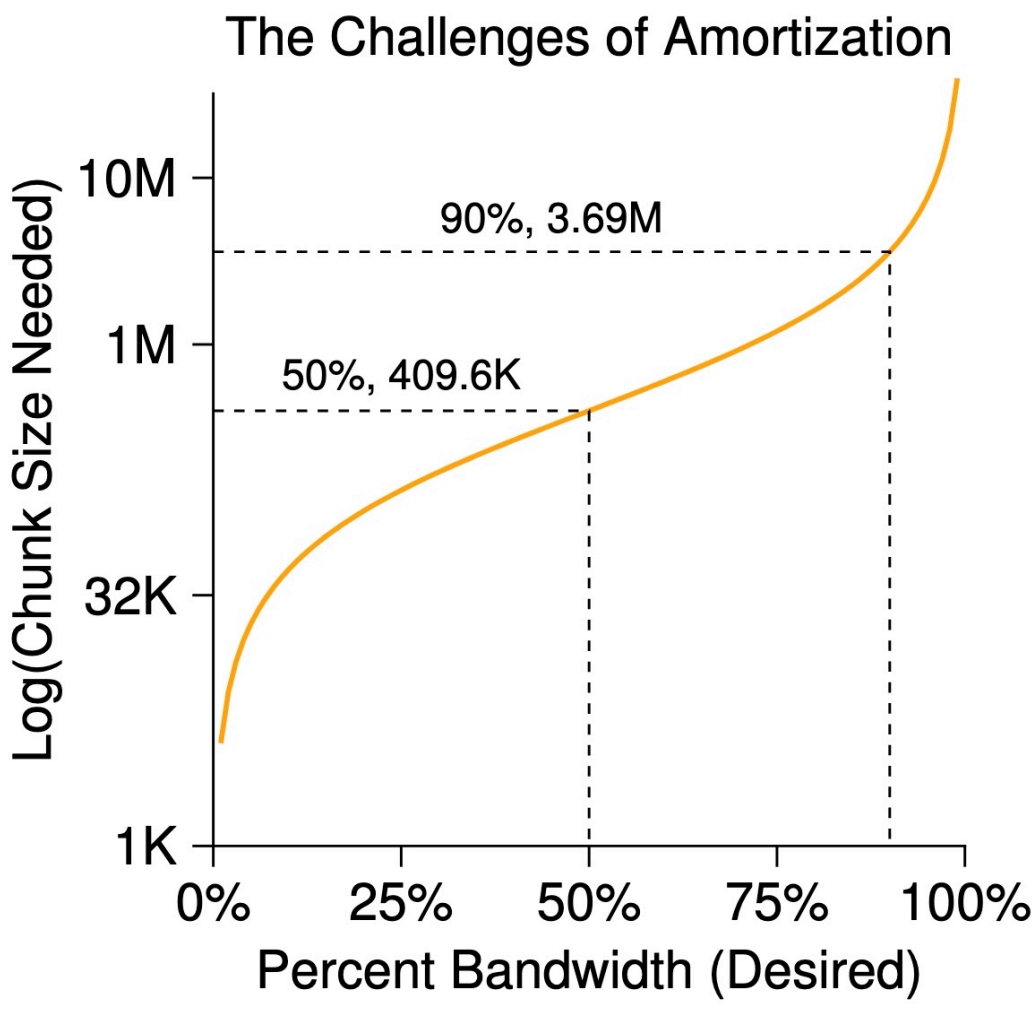


Figure 41.2: **Amortization: How Big Do Chunks Have To Be?**

FFS took a simple approach based on inode structure. The first twelve blocks were placed in the same group as the inode; each subsequent indirect block, and all the blocks it pointed to, was placed in a different group.

A Few Other Things About FFS

- To accomodate small files & reduce internal fragmentation, FFS uses sub-blocks, which were 512 byte little blocks that the file system could allocate to files.

Thus, a small (1kB) file would occupy two sub-blocks. As the file grew, the file system will continue allocating 512-byte blocks to it until it acquires a full 4kB of data.

At that point, FFS will find a 4kB block, copy the sub-blocks into it, and free the sub-blocks for future use.

This process is inefficient, requiring alot of extra work for the file system. Thus, FFS avoided this by buffering writes, and then issue them in 4kB chunks to the file system, thus avoiding the sub-block specialization entirely in most cases.

- A second neat thing that FFS introduced was a disk layout that was optimized for performance. In those times (before SCSI and other more modern device interfaces), disks were much less sophisticated and required the host CPU to control their operation in a more hands-on way. A problem arose in FFS when a file was placed on consecutive sectors of the disk, as on the left in Figure 41.3. In particular, the problem arose during sequential reads. FFS would first issue a read to block 0; by the time the read was complete, and FFS issued a read to block 1, it was too late: block 1 had rotated under the head and now the read to block 1 would incur a full rotation. FFS solved this problem with a different layout, as you can see on the right in Figure 41.3. By skipping over every other block (in the example), FFS has enough time to request the next block before it went past the disk head. In fact, FFS was smart enough to figure out for a particular disk how many blocks it should skip in doing layout in order to avoid the extra rotations; this technique was called **parameterization**, as FFS would figure out the specific performance parameters of the disk and use those to decide on the exact staggered layout scheme. You might be thinking: this scheme isn't so great after all. In fact, you will only get 50% of peak bandwidth with this type of layout, because you have to go around each track twice just to read each block once. Fortunately, modern disks are much smarter: they internally read the entire track in and buffer it in an internal disk cache (often called a track buffer for this very reason). Then, on subsequent reads to the track, the disk will just return the desired data from its cache.

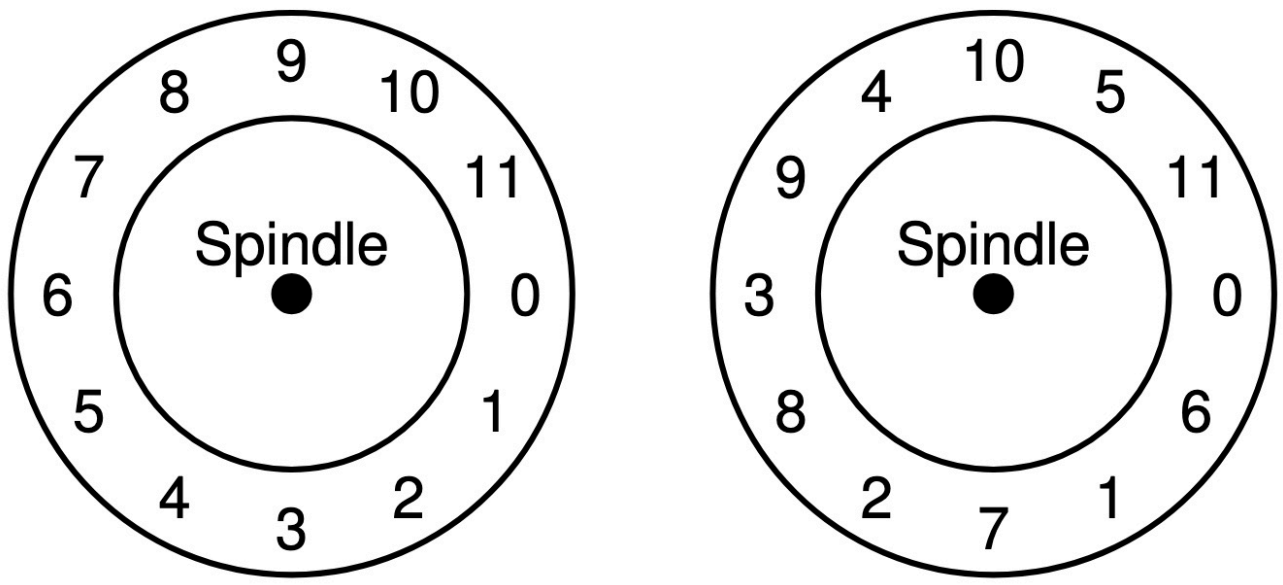


Figure 41.3: **FFS: Standard Versus Parameterized Placement**

- FFS was first to allow long file names.
- Further, symbolic links were introduced
- FFS also introduced an atomic rename() operation.